

lab1 数据包捕获与分析（利用NPcap编程捕获数据包）

才子桓 2212700 密码科学与技术

作业要求

数据包捕获与分析编程实验，要求如下：

- (1) 了解NPcap的架构。
- (2) 学习NPcap的设备列表获取方法、网卡设备打开方法，以及数据包捕获方法。
- (3) 通过NPcap编程，实现本机的数据包捕获，显示捕获数据帧的源MAC地址和目的MAC地址，以及类型/长度字段的值。
- (4) 捕获的数据包不要求硬盘存储，但应以简单明了的方式在屏幕上显示。必显字段包括源MAC地址、目的MAC地址和类型/长度字段的值。
- (5) 编写的程序应结构清晰，具有较好的可读性。

实验环境

编译IDE: Visual Studio 2022

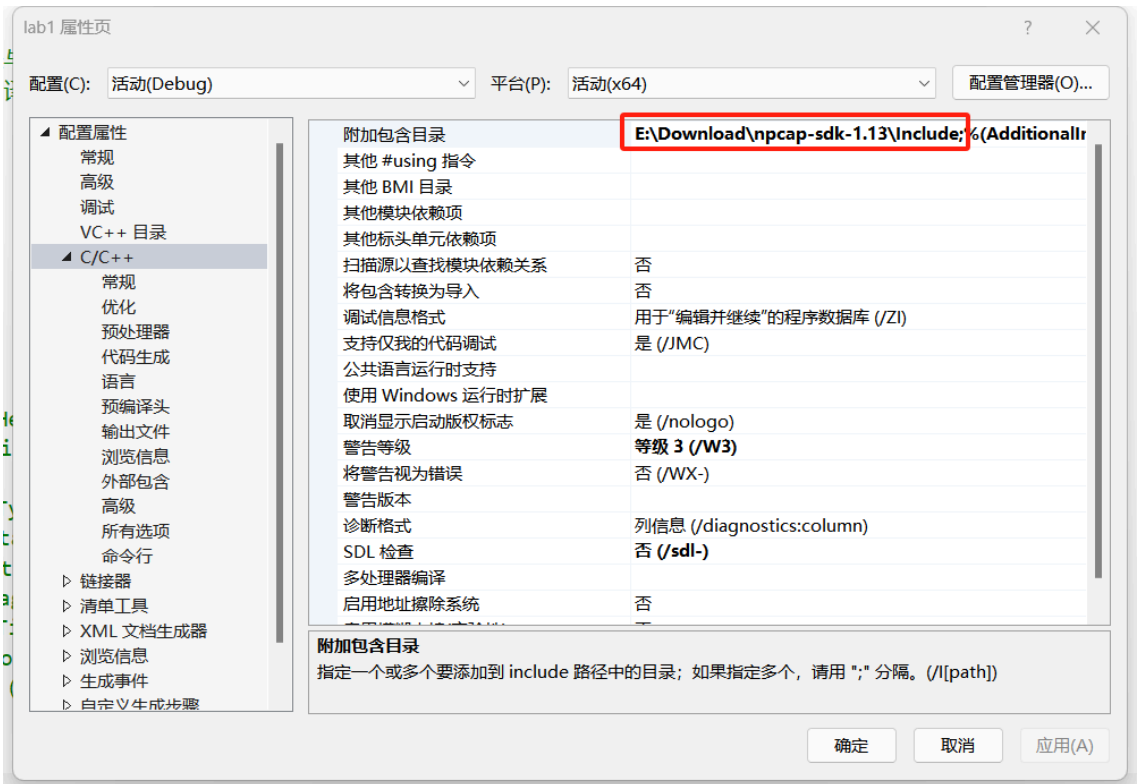
编译语言: c++ 14

其他环境: Npcap-1.80 (文件npcap-sdk-1.13)

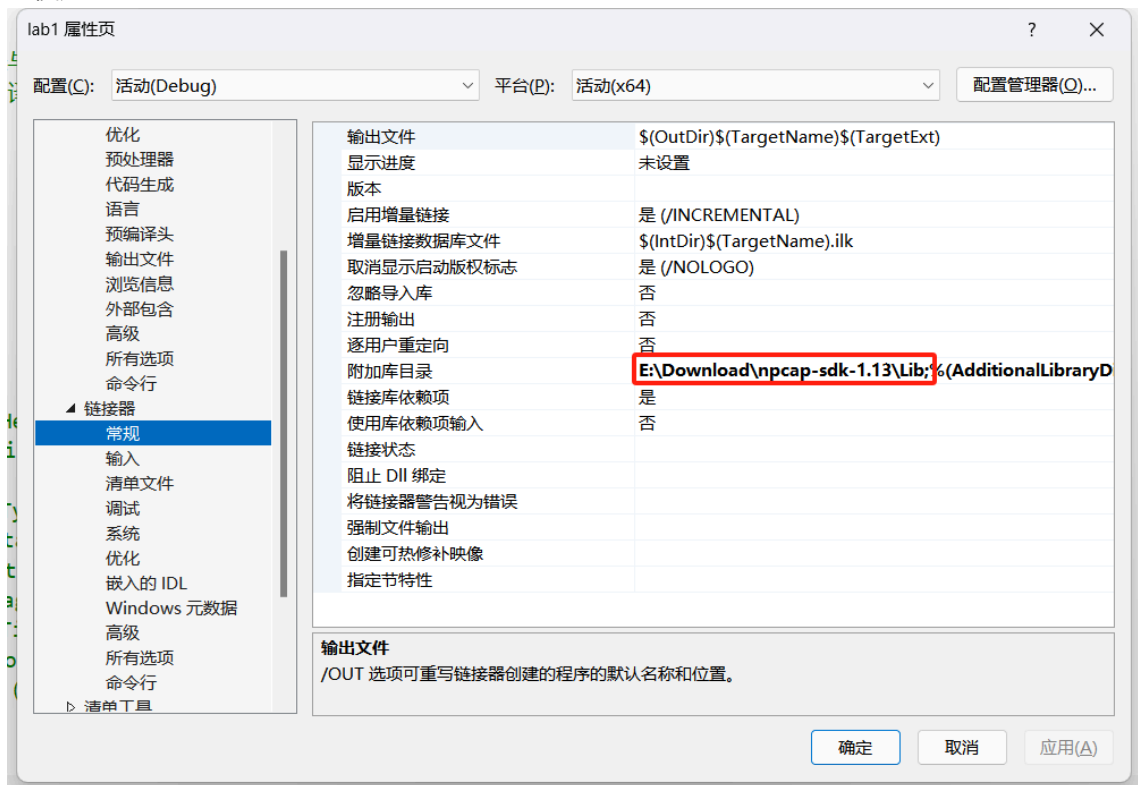
配置过程:

1. 添加包含文件目录: 添加 pcap.h 等包含文件的位置。

要在源文件 (.cpp) 前添加 `#include "pcap.h"`。

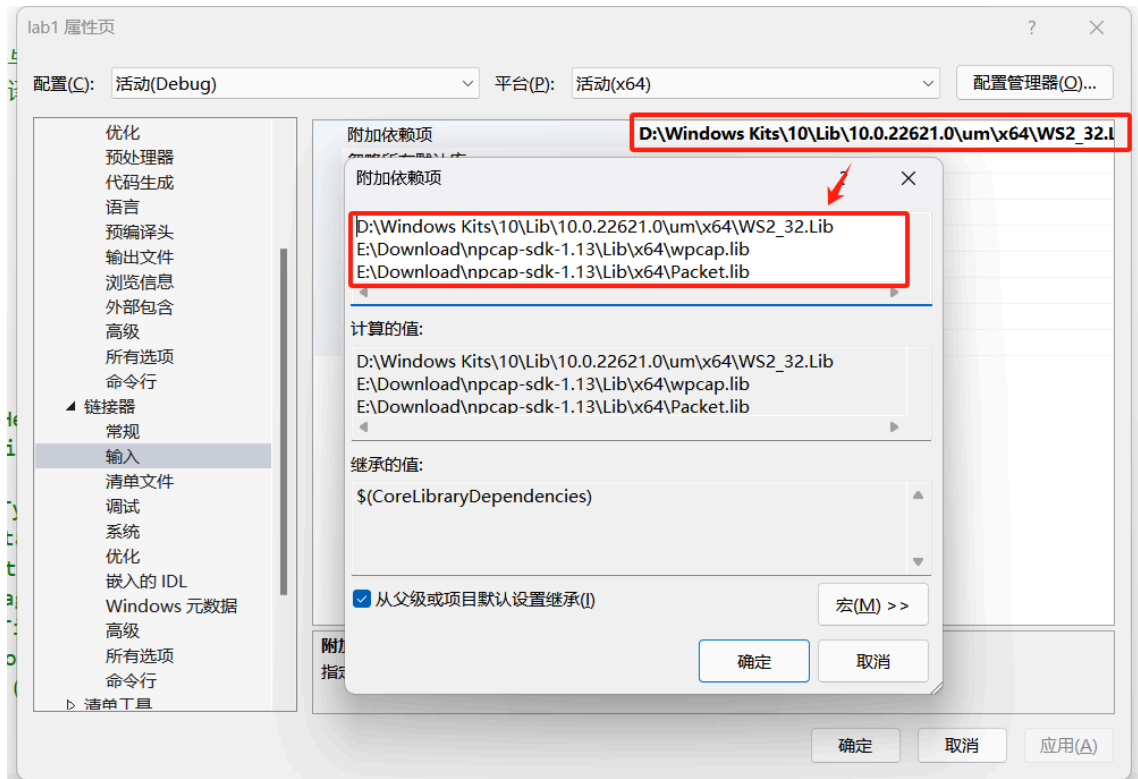


2. 添加库文件目录：在生成可执行的应用程序时，生成程序需要知道库文件所在的磁盘目录，供连接器使用。



3. 添加链接时使用的库文件：在基于 Npcap 应用程序生成过程中，需要链接 `wpcap.lib` 和 `Packet.lib` 库文件。

注意：书上提供的只需要添加 `wpcap.lib` 和 `Packet.lib` 库文件，但在实际的实现过程中会出现找不到 `pcap.h` 库的情况，所以需要添加 `windows kits` 中的 `ws2_32.Lib` 库文件。



不添加 `ws2_32.Lib` 库文件的报错信息：

LNK201: 无法解析的外部符号 <code>pcap_open_live</code> ，函数 <code>main</code> 中引用了该符号	lab1	FileName.obj	1
LNK201: 无法解析的外部符号 <code>pcap_loop</code> ，函数 <code>main</code> 中引用了该符号	lab1	FileName.obj	1
LNK201: 无法解析的外部符号 <code>pcap_freealldevs</code> ，函数 <code>main</code> 中引用了该符号	lab1	FileName.obj	1
LNK201: 无法解析的外部符号 <code>pcap_freealldevs</code> ，函数 <code>main</code> 中引用了该符号	lab1	FileName.obj	1
LNK427: 源计算机类型 'x86' 与目标计算机类型 'x64' 冲突	lab1	Packet.lib	1
LNK427: 源计算机类型 'x86' 与目标计算机类型 'x64' 冲突	lab1	wpcap.lib	1

实现思路

先定义数据帧头结构和 IP 数据包结构：

`EthernetHeader` 结构体包含：目的 MAC 地址（`dest_mac`，长度为 6 个字节的无符号字符数组）、源 MAC 地址（`src_mac`，长度为 6 个字节的无符号字符数组）和以太网类型（`ether_type`，无符号短整型）。

`ip_header` 结构体包含：IP 头部长度和版本（分别用 4 位表示）、服务类型（`ip_tos`，无符号字符型）、总长度（`total_len`，无符号短整型）、标识（`ip_id`，无符号短整型）、片偏移（`ip_off`，无符号短整型）、生存时间（`ip_ttl`，无符号字符型）、协议（`ip_protocol`，无符号字符型）、头部校验和（`ip_checksum`，无符号短整型）、源 IP 地址（`ip_source_address`，`in_addr`结构体）和目的 IP 地址（`ip_destination_address`，`in_addr`结构体）。

```
// 定义帧头结构
struct EthernetHeader {
    u_char dest_mac[6];
    u_char src_mac[6];
    u_short ether_type;
};

// 定义IP头结构
struct ip_header {
    uint8_t ip_header_length : 4,           // 头部长度（Header Length）：指示IP头部的
    长度。
    uint8_t ip_version : 4;                 // 版本（Version）：指示IP协议的版本（IPv4
    或IPv6）。
    uint8_t ip_tos;                         // 服务类型（Type of Service, ToS）：指示
    数据包的优先级和服务质量。
    uint16_t total_len;                     // 总长度（Total Length）：指示整个IP数据
    包的长度，包括头部和数据部分。
    uint16_t ip_id;                         // 标识（Identification）：用于唯一标识数
    据包，特别是在数据包分片时。
    uint16_t ip_off;                        // 片偏移（Fragment Offset）：指示数据包分
    片的位置。
    uint8_t ip_ttl;                         // 生存时间（Time to Live, TTL）：指示数据
    包在网络中的生存时间，防止数据包无限循环。
    uint8_t ip_protocol;                    // 协议（Protocol）：指示数据包携带的上层协
    议（如TCP、UDP）。
    uint16_t ip_checksum;                   // 头部校验和（Header Checksum）：用于校验
    IP头部的完整性。
    struct in_addr ip_source_address;       // 源IP地址
    struct in_addr ip_destination_address; // 目的IP地址
};
```

在主函数中利用函数 `pcap_findalldevs(&all_devices, error_buffer)` 查找本机的所有网卡，`for` 循环列出查找到的本机所有网卡，根据用户输入选择捕获哪个网卡上的数据包。

```
// 查找设备列表
if (pcap_findalldevs(&all_devices, error_buffer) == -1) {
    cerr << "Error finding devices: " << error_buffer << endl;
    return -1;
}
```

```
// 列出所有设备并编号
pcap_if_t* device;
cout << "Available network devices:" << endl;
for (device = all_devices; device; device = device->next) {
    cout << ++device_count << ". " << device->name;
    if (device->description) {
        cout << " (" << device->description << ")";
    }
    cout << endl;
}

if (device_count == 0) {
    cerr << "No devices found!" << endl;
    return -1;
}
```

通过 `device->name` 指定网卡后，用 `pcap_open_live` 打开所指定的网卡设备，`pcap_loop` 函数开始捕获指定网卡的数据包。捕获结束后函数 `pcap_freealldevs` 释放设备列表，`pcap_close` 关闭打开的网卡。

```
handle = pcap_open_live(device->name, BUFSIZ, 1, 1000, error_buffer);
if (handle == nullptr) {
    cerr << "Could not open device: " << error_buffer << endl;
    pcap_freealldevs(all_devices);
    return -1;
}

// 开始捕获
pcap_loop(handle, packets_num, packet_handler, nullptr);

pcap_freealldevs(all_devices);
pcap_close(handle);
```

这是用于处理数据包的捕获。`print_packet_details` 和 `print_packet_content` 是之前定义的打印以太网帧头部数据的内容和相关信息。捕获到以太网帧头 `eth_header` 后进行输出。

```
void packet_handler(u_char* user, const struct pcap_pkthdr* header, const u_char* packet) {
    print_packet_details(header);
    print_packet_content(packet, header->len);

    // 获取以太网帧头
    const EthernetHeader* eth_header = (EthernetHeader*)packet;

    cout << "Source MAC: ";
    print_mac_address(eth_header->src_mac);
    cout << "Destination MAC: ";
    print_mac_address(eth_header->dest_mac);

    cout << "Type: 0x" << hex << ntohs(eth_header->ether_type) << dec << "(";
    switch (ntohs(eth_header->ether_type)) {
        case 0x0800:
            cout << "IPv4";
            break;
    }
```

```

    case 0x86dd:
        cout << "IPv6";
        break;
    case 0x0806:
        cout << "ARP";
        break;
    case 0x8035:
        cout << "RARP";
        break;
    default:
        break;
}
cout << ")" << endl;

if (ntohs(eth_header->ether_type) == 0x0800) {
    const ip_header* iph = (ip_header*)(packet + sizeof(EthernetHeader));
    print_ip_header(iph); // 打印IP头部信息
}

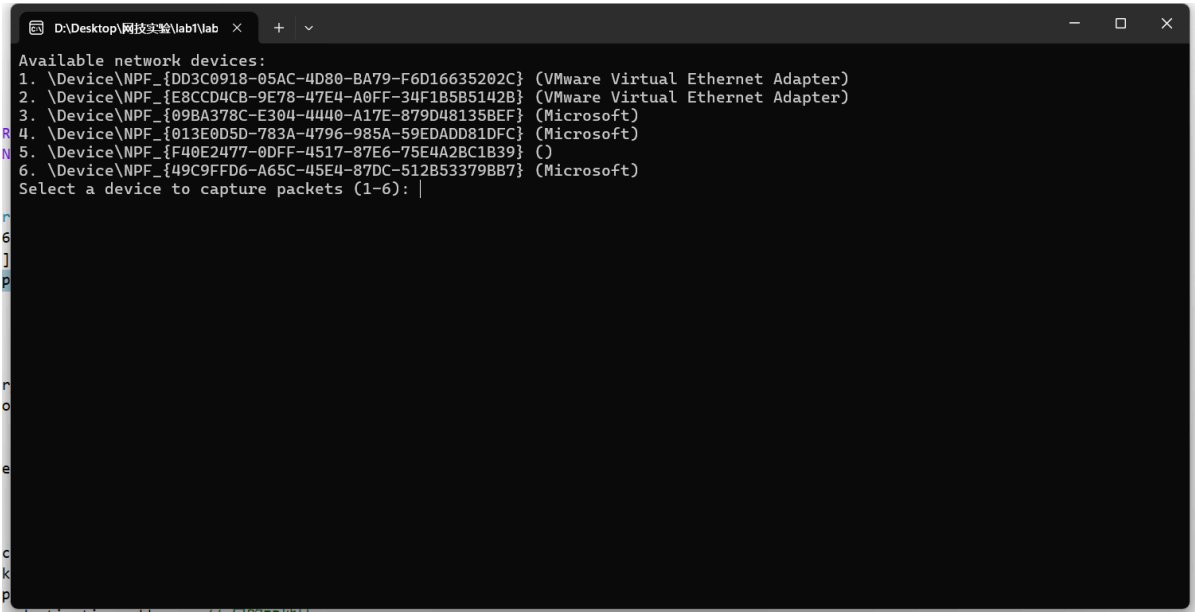
cout << "-----" << endl;
}

```

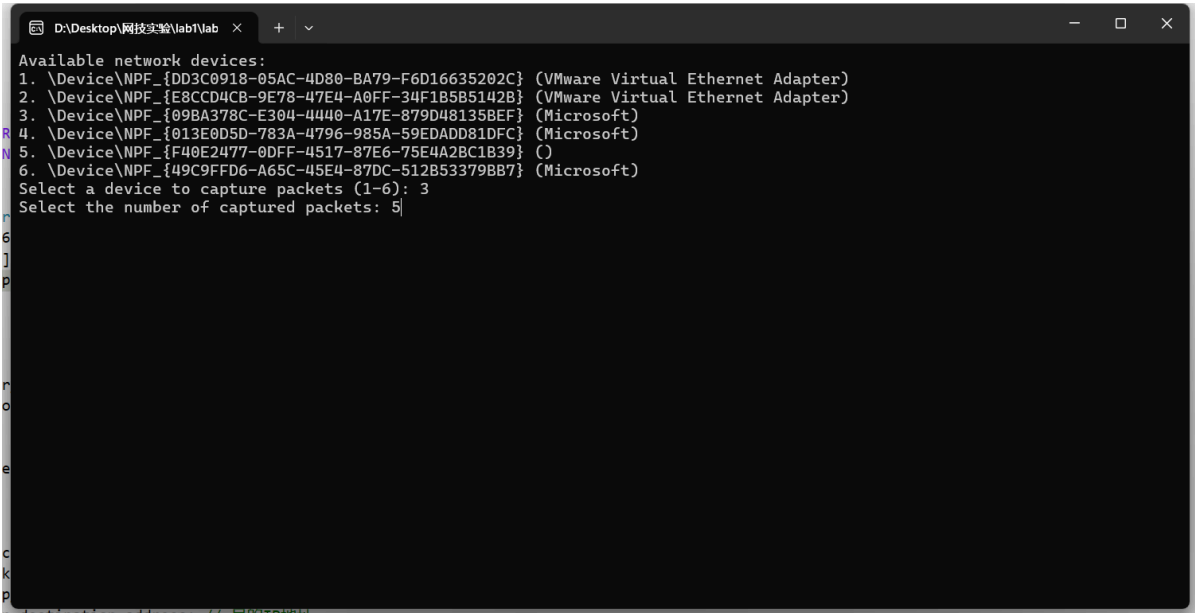
其余函数均为类似道理，输出相关信息。

效果实现

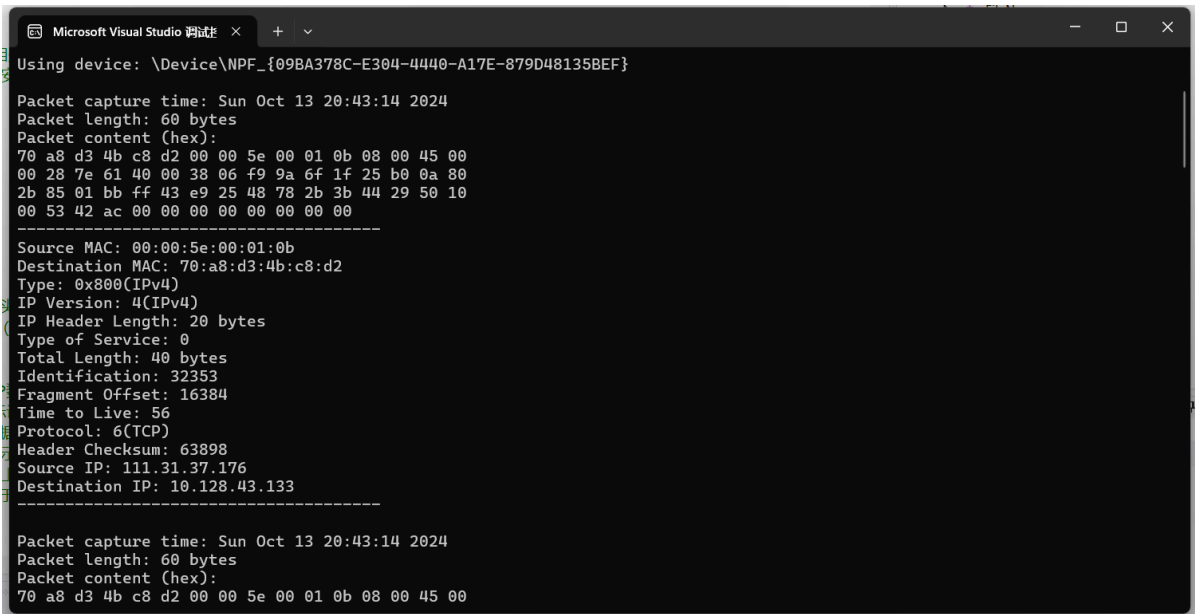
启动程序，先是查找本机上所有网卡，列出网卡名称并赋予编号，然后让用户输入监听哪个网卡。



选择网卡编号和所捕获数据包个数。



捕获数据示例：

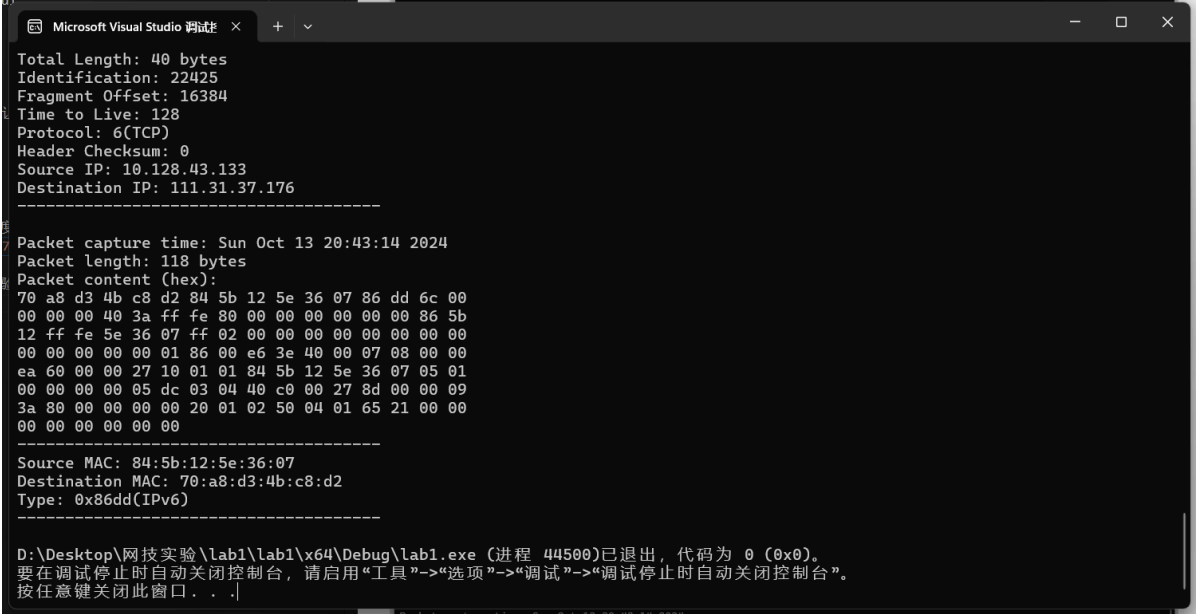


如图所示，该数据包于 sun oct 13 20:43:14 2024 被捕获，包体长度为 60 比特，内容为图中所示的

16 进制编码。源 MAC 地址为 00:00:5e:00:01:0b，目的 MAC 地址为 70:a8:d3:4b:c8:d2。所用协议为 IPv4 协议。

IP数据包头长度为 20 比特，总长度 40 比特。数据包标识号为 32353，片偏移16384，生存时间 56，头部校验和 63898。源 IP 地址为 111.31.37.176，目的 IP 地址 10.128.43.133。数据包携带的上层协议为 TCP 协议。

捕获 IPv6 协议的数据包：



```
Microsoft Visual Studio 调试
Total Length: 40 bytes
Identification: 22425
Fragment Offset: 16384
Time to Live: 128
Protocol: 6(TCP)
Header Checksum: 0
Source IP: 10.128.43.133
Destination IP: 111.31.37.176
-----
Packet capture time: Sun Oct 13 20:43:14 2024
Packet length: 118 bytes
Packet content (hex):
70 a8 d3 4b c8 d2 84 5b 12 5e 36 07 86 dd 6c 00
00 00 00 40 3a ff fe 80 00 00 00 00 00 86 5b
12 ff fe 5e 36 07 ff 02 00 00 00 00 00 00 00
00 00 00 00 00 01 86 00 e6 3e 40 00 07 08 00 00
ea 60 00 00 27 10 01 01 84 5b 12 5e 36 07 05 01
00 00 00 00 05 dc 03 04 40 c0 00 27 8d 00 00 09
3a 80 00 00 00 00 20 01 02 50 04 01 65 21 00 00
00 00 00 00 00 00
-----
Source MAC: 84:5b:12:5e:36:07
Destination MAC: 70:a8:d3:4b:c8:d2
Type: 0x86dd(IPv6)
-----
D:\Desktop\网技实验\lab1\lab1\x64\Debug\lab1.exe (进程 44500)已退出，代码为 0 (0x0)。
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口...|
```

实验反思

本次实验我们尝试使用 Npcap 制作一个数据包抓取分析程序。

在程序实现过程中，我遇到的第一个问题是环境配置问题，即我在 实验环境 中提到的问题，由于在链接器输入中附加依赖项没有添加 ws2_32.Lib 的目录导致程序出现无法运行的情况。

第二个小问题是在输出数据包内容时应该选择什么格式。在比对十六进制和字符型后，我发现字符型很多地方是无法输出的，最后选择了十六进制输出。

第三个是我在实验过程中思考 IPv6 协议下的 IP 数据包是否可以按照 IPv4 协议下的格式输出，在尝试与搜集相关资料的情况下发现这两个协议下数据包不同，由于还有其他协议的数据包且格式均不相同，所以在实验中我仅仅实现了 IPv4 协议下的数据包分析，其他协议的均未实现。